

###

自然语言处理 | 2024.1.6 考试回忆

∴ 第一题：(10)

段落分词，标出命名实体

五道问答题 (10*5)

1. 大模型为什么会产生幻觉？检索增强为什么能缓解幻觉？检索增强可以完全解决幻觉吗？
2. 为大模型提供多个示例为什么可以在不改变模型参数的情况下提升模型性能
3. GPT的训练任务是什么？现在的大语言模型为什么不使用BERT架构？
4. 大模型预训练数据清洗步骤有哪些？
5. 如何训练一个大模型完成语义角色标注任务？

计算题：(10*2)

1. BPE计算（空格也算字符哦），写出四轮计算的合并结果
2. 用单精度浮点数存储，一共有30000个词汇，一个token用1000维向量存储，那么词汇表占用空间多少？假设模型输入1个token，QKV矩阵都为1000*1000，那么三个矩阵对输入的转换一共需要多少计算？

简答题：(20)

大模型的人类反馈对齐通常使用英文数据对齐，对于一些稀疏数据的语言，大模型可能会回答一些危险问题。提出一种解决方法，在不损害英文能力的情况下也对齐其他语言，写出算法步骤和核心思想。

焚诀

建模语言 离散到连续实数空间

n元文法 rnn lstm（多一点的rnn）垃圾 transformer 牛（并行建模所有历史）

cnn为什么做不了语言模型？

3d并行 0冗余优化器

gpt bert预训练模型

提示词到指令微调+下一词预测，生成能力的通用模型

rag

考试

一. 段落分词，标出命名实体

命名实体：

- 通常指：人名 地名 单位机构名 日期 数字 货币 数量
- 在特定领域，如医学领域，有时也包含专业术语，如疾病名称、药物名称、化学成分等

命名实体识别（named entity recognition, NER）被简称为NER任务。

分词例子：

◆ 尝试：对下面的文字进行词语切分，并标注词性

克拉伦斯·威姆斯（Clarence Weems）和张宁相继上篮得手，两队比分交替上升。暂停回来，张宁和费尔德连中2记三分，88-88平！

•

克拉伦斯/nrg ·/x 威姆斯/nrf (/wkz Clarence/nrg Weems /nrf) /wky 和/c 张/nrf 宁/nrg 相继/d 上篮/v 得手/v , /wd 两/m 队/q 比分/n 交替/d 上升/v 。 /wj 暂停/v 回来/v , /wd 张/nrf 宁/nrg 和/c 费尔德/nrg 连/d 中/v 2/m 记/v 三/m 分/q , /wd 88/m -/wp 88/m 平/a ! /wt

- 切分原则12

- **切分原则1**：有明显分隔符标记的应该切分之。

分隔标记指标点符号或一个词。如：

上、下课 → 上/ 下课

洗了个澡 → 洗/ 了/ 个/ 澡

- **切分原则1**：有明显分隔符标记的应该切分之。

分隔标记指标点符号或一个词。如：

上、下课 → 上/ 下课

洗了个澡 → 洗/ 了/ 个/ 澡

- 合并原则123



1. 分词要点

- **合并原则1**：附着性语(词)素与前后词合并为一个单位。

例如：“吝”是一个附着语素，“不吝”、“吝于”等合并成一个词；“员”：检查员、邮递员、技术员等；“化”：现代化、合理化、多变化、年轻化、民营化等。

- **合并原则2**：使用频率高或共现率高的字串尽量合并。

如：“进出”、“收放”（**动词并列**）；“大笑”、“改称”（**动词偏正**）；“关门”、“洗衣”、“卸货”（**动宾结构**）；“春夏秋冬”、“轻重缓急”、“男女”（**并列结构**）；“象牙”（**名词偏正**）；“暂不”、“毫不”、“不再”、“早已”（**副词并列**）等。

- **合并原则3**：双音节加单音节的偏正式名词合并成一个分词单位。

如：“线、权、车、点”等所构成的偏正式名词，如：“国际线、分数线、贫困线”、“领导权、发言权、知情权”、“垃圾车、交通车、午餐车”、“立足点、共同点、着眼点”等。

NLP-Chapter 4.1

15



1. 分词要点

- **合并原则4**：双音节结构的偏正式动词应尽量合并。

这条原则只适合于少数偏正式的动词，如：“紧追其后”、“组建完成”等，不适合动宾及主谓式复合动词。

二.问题简答

cnn为什么做不了语言模型？

CNN（卷积神经网络）不是完全“做不了”语言模型，而是在构建现代高性能语言模型方面存在根本性局限，因此不适合替代以 Transformer 为代表的大语言模型（LLM）架构。下面从原理、结构和任务需求三个层面解释原因：

一、核心问题：CNN 的“局部性” vs 语言的“长距离依赖”

- CNN 的本质：通过滑动小窗口（卷积核）提取局部特征，强调空间/时间上的邻近关系。

- 语言的特点：句子中词与词之间的语义关系常常跨越很长距离，CNN 的短板：即使堆叠多层，感受野（receptive field）增长有限，难以有效建模这种任意长度的上下文依赖。而 Transformer 的自注意力机制可以直接关联任意两个词，无论距离多远。

二、缺乏动态上下文建模能力（无法构建动态词向量）

- Transformer 的优势：每个词的表示会根据整个句子动态调整（比如“苹果”在“吃苹果”和“买苹果股票”中含义不同）。
- CNN 的局限：卷积核是固定的，对所有位置使用相同的权重，无法根据上下文动态调整词的语义表示。

三、顺序信息处理能力弱

自然语言具有强时序性（sequential nature）。RNN 天然按时间步处理序列，能保留历史状态。

CNN 是并行、局部、平移不变的操作，虽然可以用位置编码等方式引入顺序信息，但不如 RNN 或 Transformer 中的位置机制自然

四、自回归生成困难

语言模型通常需要自回归生成（即逐词生成，每一步依赖之前所有生成的词）。

CNN 在训练时可以并行处理整个句子，但在推理（生成）阶段难以高效实现自回归，因为要防止“信息泄露”（不能看到未来词），需使用 mask 和复杂的因果卷积（如 WaveNet 中的做法），但效率和灵活性仍不如 Transformer 的 attention。

n元文法原理与缺陷

n元文法（n-gram model）是统计语言模型中最经典、最基础的方法之一，其核心思想是：一个词出现的概率只依赖于它前面的 n-1 个词。

• 主要缺陷

1. 数据稀疏问题（Sparsity）

- 语言组合爆炸：即使中等长度的句子，其 n-gram 在训练集中很可能从未出现过。
- 导致大量 n-gram 的频次为 0，概率估计为 0 → 整个句子概率为 0，无法使用。

即使平滑只是“修补”，不能根本解决泛化能力弱的问题。

2. 上下文窗口太短（有限历史）

- n 元文法只能看到最近的 n-1 个词。
- 无法捕捉长距离依赖。例如：

“The cat, which ate the fish that was on the table, was full.”

动词 “was” 要与主语 “cat”（单数）一致，但中间隔了多个从句——bigram/trigram 无法建模。

3. 缺乏泛化能力（无语义理解）

- n-gram 是纯粹的表面形式匹配，不理解词义或句法结构。
- 无法处理同义替换或结构变化。例如：
 - 训练语料有 “buy a book”
 - 测试时遇到 “purchase a novel” → 概率极低，尽管语义相近。

4. 也缺乏动态上下文建模能力（无法构建动态词向量）

3D并行 零冗余优化器 (ZeRO)

“3D并行 + 零冗余优化器 (ZeRO)”是当前大规模分布式训练大语言模型 (LLM) 的核心技术组合，旨在解决模型参数、梯度、优化器状态等在多GPU/多节点间高效、低冗余分配的问题。下面系统讲解其原理与协同机制。

一、背景：为什么需要 3D 并行 + ZeRO？

训练千亿级参数模型（如 GPT-3、LLaMA）时，单卡显存远远不够。传统数据并行（Data Parallelism）会在每个 GPU 上完整复制模型、梯度和优化器状态，造成严重内存冗余和通信开销。

为突破这一瓶颈，业界发展出“3D 并行”+“ZeRO”的混合策略：

- 3D 并行：从三个维度切分计算与通信
- ZeRO (Zero Redundancy Optimizer)：消除优化器状态、梯度、参数的冗余存储

二者结合，可将显存占用降低一个数量级，支持超大规模模型训练。

二、3D 并行：三种并行方式的组合

“3D”指以下三种并行策略的正交组合：

1. 数据并行 (Data Parallelism, DP)

- 每个设备持有一份完整模型副本
- 各自处理不同 mini-batch 数据
- 前向/反向后，同步梯度 (All-Reduce)
- ☒ 优点：简单；☒ 缺点：模型全复制，显存浪费

2. 张量并行 (Tensor Parallelism, TP)

(又称模型并行 / 算子内并行)

- 将单个算子（如 Linear 层）的权重矩阵按行或列切分到多个 GPU
- 计算时通过 All-Gather / Reduce-Scatter 协同完成矩阵乘法
- 例如： $Y = XW$ ，将 W 切分为 W_1, W_2 ，各 GPU 计算 XW_1, XW_2 ，再拼接
- ☒ 减少单卡参数量；☒ 增加设备间通信（每层都要通信）

典型实现：Megatron-LM (NVIDIA)

3. 流水线并行 (Pipeline Parallelism, PP)

- 将模型按层切分到不同设备（如 GPU 0 负责 layer 1-10，GPU 1 负责 11-20）
- 输入数据像“流水线”一样逐级传递
- 使用 micro-batch 技术提高设备利用率（避免空闲等待）
- ☒ 减少单卡显存；☒ 存在 bubble（空闲时间），需调度优化

典型实现：GPipe、PipeDream

🔥 3D 并行 = DP + TP + PP

例如：64 个 GPU 可配置为 DP=4, TP=2, PP=8 ($4 \times 2 \times 8 = 64$)

三、零冗余优化器 (ZeRO)：消除冗余存储

由 Microsoft 提出，核心思想：不复制，只存自己负责的部分。

ZeRO 分为三个阶段 (Stage)：

阶段 冗余消除对象 显存节省

ZeRO-1 优化器状态 (如 Adam 的 momentum、variance) ~50%

ZeRO-2 + 梯度 (Gradients) ~75%

ZeRO-3 + 模型参数 (Parameters) ~90%+

ZeRO-3 工作流程 (最关键) :

1. 前向计算前: 通过 all-gather 临时收集完整参数 (仅当前层所需)
2. 计算完成后: 立即释放其他设备的参数, 只保留自己负责的 shard
3. 反向传播时: 同样动态 gather 参数, 计算梯度后 scatter 回各自 shard
4. 优化器更新: 每个 GPU 只更新自己负责的那部分参数

⚠ ZeRO-3 引入额外通信开销 (频繁 all-gather/scatter), 但换来巨大显存节省。

四、3D 并行 + ZeRO 如何协同?

实践中, ZeRO 通常作用于数据并行维度 (DP) 内部, 而 TP 和 PP 在更细粒度上切分模型。

典型配置 (以 DeepSpeed + Megatron 为例) :

- 总 GPU 数 = $DP \times TP \times PP$
- 在每个 DP 组内, 启用 ZeRO (如 ZeRO-3)
- TP 和 PP 负责模型结构切分
- ZeRO 负责在 DP 组内去冗余存储优化器状态、梯度、参数

✅ 优势:

- 显存占用极低 (可训 1T+ 参数模型)
- 通信效率高 (TP/PP 减少跨节点通信, ZeRO 优化 DP 内通信)

五、实际框架支持

框架 支持能力

DeepSpeed (Microsoft) 完整 ZeRO-1/2/3 + 与 Megatron 集成支持 TP/PP

Megatron-LM (NVIDIA) 强大的 TP + PP, 可与 DeepSpeed ZeRO 结合

Colossal-AI 支持 2D/3D 并行 + ZeRO-like 优化

PyTorch FSDP (Fully Sharded Data Parallel) 类似 ZeRO-3 的原生实现

六、总结

技术 解决问题 关键思想

数据并行 (DP) 扩展 batch size 复制模型, 同步梯度

张量并行 (TP) 单层太大放不下 切分权重矩阵

流水线并行 (PP) 模型层数太多 按层分片, 流水执行

ZeRO 冗余存储浪费显存 只存自己负责的 shard

🔑 3D 并行 + ZeRO = 超大规模模型训练的“黄金组合”

它让千卡集群能高效训练万亿参数模型, 是 LLM 工业化的基石技术。

如果你在部署大模型训练, 理解这套架构对资源规划、性能调优至关重要。

1. 大模型为什么会产生幻觉? 检索增强为什么能缓解幻觉? 检索增强可以完全解决幻觉吗?

答:

- 产生幻觉的原因:
 - 大模型是基于大规模语料训练的自回归模型, 其输出依赖于概率预测, 而非真实世界事实。

- 模型可能生成看似合理但不真实的陈述（如虚构人物、事件或数据），称为“幻觉”。
- 幻觉主要源于以下几点：
 - 缺乏外部知识验证机制；
 - 训练数据中存在噪声或矛盾信息；
 - 模型倾向于生成流畅、连贯但不一定准确的内容。
- 检索增强如何缓解幻觉：
 - 检索增强（Retrieval-Augmented Generation, RAG）在生成前从外部知识库（如维基百科、数据库）中检索相关文档片段作为上下文输入。
 - 这样模型生成时可参考真实、可靠的信息，从而减少凭空捏造的可能性。
 - 例如：当用户问“爱因斯坦出生年份”，RAG会先检索到“1879年”，再据此生成答案，避免错误。
- 能否完全解决幻觉？
 - 不能。虽然RAG显著降低了幻觉率，但仍存在局限性：
 - 检索结果本身可能不准确；
 - 模型仍可能对检索结果进行错误解读或推理；
 - 对于复杂推理任务或长尾问题效果不佳。

2. 为大模型提供多个示例为什么可以在不改变模型参数的情况下提升模型性能？

答：

这种技术称为提示学习（Prompt Learning），核心思想如下：

- 模型在训练阶段已学习了语言模式和上下文理解能力。
- 提供多个示例相当于向模型展示了任务格式、输入输出结构和期望行为。
- 模型利用其强大的泛化能力，模仿这些示例中的模式来完成新任务。

原因解释：

- 模型并非“记忆”示例，而是通过上下文感知提取任务特征；
- 示例提供了语义线索和逻辑结构，引导模型激活正确的内部表示；
- 不需要微调参数，因为模型已经具备足够的容量去适应不同任务。

关键点：多示例提示是一种“软监督”方式，利用上下文信息激发模型已有知识，实现零样本/少样本推理。

3. GPT的训练任务是什么？现在的大语言模型为什么不使用BERT架构？

答：

- GPT的训练任务：
 - GPT系列采用自回归语言建模（Autoregressive Language Modeling）。
 - 目标：给定前面的token序列，预测下一个token的概率分布。
 - 数学形式：最大化 $P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$
- 为何现代大模型不用BERT架构？

维度 BERT 现代LLM（如GPT）

架构 双向Transformer（编码器） 单向Transformer（解码器）

训练目标 掩码语言建模（MLM） 自回归语言建模（ARLM）

输出能力 仅支持分类、填空等有限任务 支持生成式任务（写作、对话、代码等）

应用场景 理解类任务为主 生成类任务为主

主要原因：

1. 生成能力弱：BERT只能做理解任务，不能生成连续文本；
2. 缺乏因果建模：双向注意力导致未来信息泄露，不适合生成；
3. 训练效率低：MLM需要掩码策略，难以扩展到超大规模；
4. 下游任务适配差：需大量fine-tuning，而GPT可通过prompt直接使用。

结论：尽管BERT在理解任务上表现优异，但现代大模型追求通用生成能力，因此普遍采用类似GPT的自回归架构。

4. 大模型预训练数据清洗步骤有哪些？

答：

高质量的数据是大模型成功的关键。典型清洗流程包括：

1. 格式标准化
 - 清理HTML标签、特殊符号、换行符等，统一文本格式。
2. 去重 (Deduplication)
 - 移除重复网页、文档或句子，防止模型过拟合。
3. 过滤低质量内容
 - 基于长度、字符频率、HTML标签密度判断是否为垃圾内容（如广告、爬虫日志）。
4. 毒性与敏感内容过滤
 - 使用规则或分类器识别仇恨言论、色情、暴力等内容。
5. 版权与隐私保护
 - 过滤受版权保护的内容（如书籍、新闻稿）、个人身份信息 (PII) 。
6. 语言识别与过滤
 - 使用语言检测工具（如langdetect）筛选目标语言（如中文/英文），去除混合或多语言文本。

5. 如何训练一个大模型完成语义角色标注任务？

答：

语义角色标注 (Semantic Role Labeling, SRL) 是识别句子中谓词及其论元的角色（如施事、受事、时间等）的任务。

方法一：有监督微调 (Supervised Fine-tuning)

1. 准备标注数据
 - 获取带SRL标注的数据集（如PropBank）。
 - 将每个句子转化为输入-输出对。
2. 设计输入格式
 - 输入：[CLS] 主语 谓词 宾语 [SEP]
 - 输出：: 主语 : 宾语
3. 微调模型
 - 在预训练模型（如BERT、RoBERTa）基础上，添加分类头或序列标注层。
 - 使用交叉熵损失函数训练。

4. 评估指标

- 使用精确率、召回率、F1值衡量效果。

方法二：提示学习 (Prompt-based)

- 设计模板：
句子：小明吃了苹果。
请标注语义角色：
施事：__
动作：__
受事：__
- 模型根据提示生成答案，无需额外参数更新。

方法三：指令微调 (Instruction Tuning)

- 将SRL任务表述为自然语言指令：
“请找出这句话中的主语、谓语、宾语，并标记它们的语义角色。”
- 在指令数据集上进行微调，使模型学会遵循指令执行任务。

推荐方案：对于资源充足情况，采用监督微调 + 指令微调结合的方式；若数据稀缺，则优先尝试提示学习。

三.简单计算

1. BPE计算（空格也算字符哦），写出四轮计算的合并结果

假设输入文本为："the cat sat on the mat"

我们按BPE算法逐步合并。

步骤1：初始词汇表

所有字符和空格组成基本单元：

[`'t', 'h', 'e', ' ', 'c', 'a', 't', ' ', 's', 'a', 't', ' ', 'o', 'n', ' ', 't', 'h', 'e', ' ', 'm', 'a', 't'`]

步骤2：统计频次

- t: 4次
- h: 2次
- e: 2次
- : 4次
- c: 1次
- a: 3次
- s: 1次
- n: 1次
- m: 1次

token 初始为: [`"t","h","e"," ","c","a","t"," ","s","a","t"," ","o","n"," ","t","h","e"," ","m","a","t"`]

统计相邻对频次：

组合 频次

t h 2

h e 2

e 3 (后面接空格)

c 1

c a 1

a t 2

t 3 (前面是空格)

s 1

s a 1

a t 2

t 3

o 1

o n 1

n 1

t 2

t h 2

h e 2

e 3

m 1

m a 1

a t 2

最高频组合: a t → 出现2次 ("cat", "sat", "mat")

第1轮合并: 将 a t 替换为 at

→ 新序列: ["t","h","e"," ","c","at"," ","s","at"," ","o","n"," ","t","h","e"," ","m","at"]

...

2. 用单精度浮点数存储, 一共有30000个词汇, 一个token用1000维向量存储, 那么词汇表占用空间多少? 假设模型输入1个token, QKV矩阵都为1000*1000, 那么三个矩阵对输入的转换一共需要多少计算?

答:

(1) 词汇表占用空间

- 每个词嵌入维度: 1000
- 每个浮点数: 4字节 (单精度)
- 总词汇数: 30000

text{总大小} = 30000 \times 1000 \times 4 \text{ text{ 字节}} = 120,000,000 \text{ text{ 字节}} = 120 \text{ text{ MB}}

答案: 120 MB

(2) QKV矩阵对输入的计算量

- 输入 token: 1000维向量
- QKV 矩阵均为 1000 times 1000
- 每个矩阵乘法: $1000 \times 1000 = 10^6$ 次浮点运算 (FLOPs)

- 三个矩阵: Q, K, V 各一次
nm矩阵和mp矩阵相乘, 需要 $2 \times nmp$ 次浮点运算

text{总计算量} = 3 * 2 \times 10^6 = 6 \times 10^6 \text{ text{ FLOPs}}

答案: 3百万次浮点运算 (6×10^6 FLOPs)

四.方法设计

爬完数据怎么处理 分类器 (统计方法或者神经网络方法 但是要合理)

(第一步 训练数据怎么标 表什么 第二步。。。)

问题: 大模型的人类反馈对齐通常使用英文数据对齐, 对于一些稀疏数据的语言, 大模型可能会回答一些危险问题。提出一种解决方法, 在不损害英文能力的情况下也对齐其他语言, 写出算法步骤和核心思想。

答:

核心思想:

利用跨语言迁移学习与多语言安全对齐机制, 在保持英文性能的同时, 通过少量高质量非英语数据进行安全对齐。关键是避免灾难性遗忘, 同时实现语言间的正迁移。

算法步骤

1. 目标语言理解与安全判断

对任意非英语输入 (如斯瓦希里语、缅甸语等):

使用多语言编码器 (如mBERT、XLM-R) 将用户输入编码为语义向量, 并在目标语言空间中进行安全判断:

- 检测是否包含本地化有害内容 (如宗教冒犯、地域歧视等);
- 若判定为高风险, 直接拦截或返回通用安全模板。

2. 英文生成 & 回译:

若通过安全检测, 则将用户query翻译成英文提示, 送入已对齐的英文大模型 (如经过RLHF的Llama-3-English) 生成安全、合规的英文回答;

再通过高质量神经机器翻译 (如NLLB、mBART等) 系统将英文回答回译为目标语言。

3. 训练两个阶段的分类器, 以更好的性能满足下游任务:

- 不微调主模型: 保持英文大模型参数冻结, 避免破坏其已有的安全对齐能力。
- 仅训练两个轻量模块:
 - 一个多语言安全分类器 (基于XLM-R, 在少量多语言有害样本上微调);
 - 一个指令分析模块 (如NLLB、mBART等)

卷面干净 可以乱讲 但是要自圆其说

其他一些重要考点

9微调

指令微调

基于人类反馈学习

10提示学习

基础提示

上下文学习

思维链

11多语言多模态

多语言方法

多模态方法

12rag

对于模型出来之后的事情只能检索增强

14信息抽取

信息：实体、概念以及他们之间的关系

事件抽取：实施者 受试者承受者

15机器翻译

序列：输入和输出之间的对应关系

统计学习方法：短语翻译模型（短语表+语言模型+对齐模型）

神经网络方法：端到端（rnn，对每一个词编码） 注意力机制（也是端到端 transformer）

译文评价：bleu （n-gram 匹配度）